

Sehat Clinic Voice Agent

An outbound healthcare voice agent on LiveKit. It calls a patient, tells them their **HbA1c and fasting glucose** results, books a consultation through a tool call, then analyses the call and ships the whole thing — transcript, audio, tool results, analysis and evaluation scores — into **Opik** through a single drop-in module.

`CODE` github.com/Mayanksingh2518/voice-livekit

AGENT LiveKit Cloud · ap-south

CONSOLE Render · free tier

TRACES Opik · livekit-voice-agent

What was built

A complete outbound calling system, not a demo script. The agent dials a patient, verifies who it is speaking to *before* disclosing anything clinical, reads results it was given rather than results it recalls, refuses medical advice, and books a consultation through a simulated scheduler. Every call — including the ones nobody answers — ends up as a scored, auditable trace.

REQUIREMENT COVERAGE

ASSIGNMENT REQUIREMENT	DELIVERED
Outbound agent on LiveKit, given name, phone and biomarkers	done <code>main.py</code> — SIP dial with per-status outcome mapping
Inform the person about their health metrics	done values supplied literally; the model may not invent or convert them
Attempt to schedule a doctor consultation	done availability search, alternatives on conflict, confirmation read back

ASSIGNMENT REQUIREMENT	DELIVERED
Tool / function call to simulate booking	done <code>services/scheduler.py</code> — deterministic slots, conflicts, horizon and lead-time rules
Post-call analysis with outcome and booking status	done structured output, reconciled against the tool record
Opik: call metadata and variables	done the prompt's bound variables are logged verbatim
Opik: conversation / transcript	done one trace per turn, sharing a <code>thread_id</code>
Opik: call recording or audio reference	done recorded in-process, attached to the trace
Opik: tool calls and results	done <code>type="tool"</code> spans with arguments and results
Opik: post-call analysis	done trace output plus its own <code>llm</code> span
At least one online evaluation	done two server-side rules (trace and thread level) plus in-process judges
Opik as a standalone, pluggable module	done one file, one line to attach, zero imports from the app
README and demonstration of the full flow	done README, a web console, and a replay script that runs the pipeline without a call
Live call to a PSTN phone	carrier code complete; blocked by free-tier telephony — see Constraints

How it fits together

Two deployables, because they are different shapes. The agent is a long-lived worker holding a socket to LiveKit; the console is an ordinary HTTP service. Putting them on the same host would have meant paying for a worker that idles.

Agent worker	The call itself — prompt, six tools, recording. Deployed to LiveKit Cloud (<code>lk agent deploy</code>), ap-south for latency.
Web console	Start a call, watch the transcript stream, read the analysis. Deployed to Render . Reads call data back out of Opik rather than keeping a copy.
<code>opik_tracer.py</code>	The deliverable module. Hooks LiveKit's session events and a shutdown callback; emits turn traces, a call trace, tool spans, the audio attachment and scores.
Scheduler	Stands in for a real booking system. Deterministic availability so a demo reproduces; conflicts return alternatives rather than failing.

THE DECISION I WOULD DEFEND FIRST

The tool record outranks the transcript. The analysis model reads what was said and forms a judgement; `CallState` records what actually happened. Where they disagree — above all on whether an appointment exists — the deterministic record wins and the disagreement is written into the trace.

A model that narrates a booking it never made cannot manufacture one. Without this, "appointment booked" is a claim rather than a fact, and the whole funnel metric becomes untrustworthy.

The Opik integration

Attaching it costs the core application one line:

```
tracer = OpikCallTracer.from_env(call_id=..., variables=...)
tracer.attach(session, ctx, finalise=build_summary)
```

- **Zero coupling.** The module imports nothing from the application. It takes a session-shaped object with an `.on()` method and one async callback returning a plain dict, so it drops into any LiveKit project unchanged.

- **Fail-open.** Collection runs whether or not Opik is configured. Disabling telemetry changes only where data goes, never whether the post-call analysis runs. A voice agent must not drop a patient because telemetry broke — and `OPIK_ENABLED=false` demonstrates it rather than asserting it.
- **One trace per turn, not one per call.** In Opik a thread *is* a set of traces sharing a `thread_id`; thread-level rules run across sibling traces, never across child spans. Logging the call as a single trace would look tidier and silently disqualify it from every thread-level evaluation.
- **Manual SDK over the OpenTelemetry integration.** Opik ships a LiveKit integration, but it is OTel-based: it cannot set a `thread_id`, carry an audio attachment, or hold post-call analysis. A considered choice, not an unawareness of it.

Running it

Three accounts, all free tier, no card required. Setup is verified by a script that calls every service rather than checking that keys are merely present.

CREDENTIALS NEEDED

KEY	WHERE FROM	COVERS
LIVEKIT_*	cloud.livekit.io → Settings → Keys	Rooms, agent runtime, SIP
GEMINI_API_KEY	aistudio.google.com/apikey	Agent replies, post-call analysis, judges
DEEPGRAM_API_KEY	console.deepgram.com	Speech to text <i>and</i> text to speech
OPIK_API_KEY	comet.com/opik	Traces, threads, evaluations

Local

```
uv venv --python 3.11 && uv sync --extra agent
```

```
cp .env.example .env — fill in the four keys above
```

```
uv run python main.py download-files — VAD and turn-detector weights
```

```
uv run python scripts/check_setup.py — calls every service, names what is missing
```

```
uv run python scripts/setup_opik_rules.py --provider-key && uv run python  
scripts/setup_opik_rules.py
```

```
uv run python main.py dev — the agent worker
```

```
uv run python web/server.py — the console, on :8080
```

- **Demo without speaking or dialling:** `uv run python scripts/replay_call.py` pushes a scripted conversation through the real post-call pipeline — analysis, reconciliation, Opik traces, tool spans and scores. It also drives the Opik module from a fake session object with no LiveKit runtime, which is the proof that the module is genuinely standalone.
- **Prove the fail-open claim:** `OPIK_ENABLED=false uv run python scripts/replay_call.py` — the analysis still completes, nothing is sent.
- **Use headphones for a browser call.** On speakers the agent hears itself and starts replying to its own voice.
- **Tests:** `uv run --group dev pytest` — 34 tests over the scheduler (conflicts, alternatives, closed days, relative dates, booking horizon) and the analysis reconciliation (the model claiming a booking that did not happen, missing one that did, and every deterministic override).

Deployed

- **Agent → LiveKit Cloud.** `lk agent create --secrets-file .env.agent`, then `lk agent deploy` for later versions. A `Dockerfile` and `.python-version` are in the repository; model weights are fetched at build time so the first call is not delayed.
- **Console → Render.** Python 3, build `pip install .`, start `python web/server.py`, free instance. Seven environment variables; the agent's speech stack is deliberately excluded so the console stays inside a 512 MB instance.

Constraints encountered

All three are account-tier limits of free services, not limits of the design. Each was diagnosed rather than assumed, and each is recorded here because a reviewer will reasonably ask why there is no recording of a ringing phone.

1 • TELEPHONY – OUTBOUND SIP IS NOT AVAILABLE ON ANY FREE TIER TESTED

Twilio trial blocks Elastic SIP Trunking outright, rejects the inline `Twiml` API parameter (*"trial accounts have limited parameter access"*), and drops `<Dial><Sip>` silently. The silence was the hard part: adding a `<Say>` before the dial stretched the call from 5s to 11s, which proves Twilio fetched and executed the markup while the SIP leg never formed. Its debugging APIs are also gated, so the error itself is unreadable.

Telnyx trial does permit SIP trunking and issues a free number — then refuses the destination: *"You must upgrade your account in order to use the following countries: IN."* North America is permitted; India is not.

The agent's outbound code is complete and unchanged by any of this. `SIP_OUTBOUND_TRUNK_ID` is the only thing standing between it and a ringing phone; a paid trunk on either provider makes the same code dial.

2 • LLM RATE LIMITS MAKE A FREE TIER UNUSABLE FOR LIVE VOICE

Gemini's free tier allows **five requests per minute** and twenty per day, per model. A voice conversation makes one LLM call per turn, so a normal exchange exhausts the per-minute quota inside a minute and the agent simply stops mid-sentence.

Mitigated by giving each role its own model — the agent, the analyser and the judges draw on separate quotas — but the real fix is a paid key. Roughly \$0.05 of usage per call would remove the ceiling entirely.

3 • FREE HOSTING CANNOT RUN A PERSISTENT WORKER

Render's background workers are paid-only, Fly.io's free tier is gone for new accounts, Koyeb's free tier excludes worker services, and Hugging Face Spaces kills anything that does not answer on an HTTP port. Binding a dummy port does not help: these platforms sleep on absent inbound traffic, and the agent's connection is outbound.

Resolved by using LiveKit Cloud's own agent hosting, which is purpose-built for this shape and includes a genuine free allowance. The console, being ordinary HTTP, sits on Render's free tier.

Bugs found by running it

Worth listing because each was invisible from reading the code, and several would have reached production.

The agent never spoke its opening line

`generate_reply(instructions=...)` sends no conversation message. OpenAI accepts that; Gemini rejects it outright. The greeting is fixed text, so it is now spoken directly — faster and unable to fail.

Traces arrived in Opik with no name

Calling `trace.end()` after already supplying `end_time` queues a second, mostly-empty update that batching merges over the original. Everything is now supplied at creation.

The agent started answering itself

Its own voice came back through the speakers, was transcribed as the patient, and it replied to that. Echo cancellation must be requested explicitly in a browser; a phone handles it in hardware.

A safety judge scored 0.1 while its own reasoning was wholly positive

G-Eval derives a continuous score from token probabilities; binary criteria ("1 if all hold, else 0") make it return near-zero. Rewritten as a graded rubric, it now scores 1.0 with matching reasoning.

Online rules were created but never scored anything

They execute on Opik's servers and need their own provider credential registered there, and the model name is Opik's own — not the LiteLLM name used by the in-process metrics. Two different fields that look identical.

Calls that failed slowly produced no trace at all

Analysis retries pushed the shutdown callback past the worker's grace period and the process was killed mid-write — losing exactly the calls most worth inspecting. The grace period now accommodates the flush.

Disabling telemetry silently broke the analysis

Collection was gated on Opik being configured, so the analyser received an empty transcript. Collection now always runs; only the destination is conditional.

Taking this to production

What I would build next, in the order I would build it.

FIRST

Make the booking real and idempotent

The scheduler holds slots in process memory. A real deployment writes to the clinic's system with an idempotency key per call, so a retried tool call cannot double-book, and a crash between "slot taken" and "confirmed" resolves in the patient's favour rather than leaving a ghost appointment.

FIRST

Campaign layer above the agent

Today a call is a single dispatch. Production needs a queue that owns retry windows, respects calling hours and the patient's timezone, caps

attempts, honours do-not-call permanently rather than per-call, and never places two calls to the same person at once. The agent stays a worker; the campaign owns the policy.

NEXT

Turn evaluation into a regression suite

Online rules catch problems after they reach patients. The same judges should run offline against a curated dataset of transcripts — including the adversarial ones, where the patient asks for a diagnosis or someone else picks up — so a prompt change is measured before it ships, not after. Opik's experiments are built for exactly this.

NEXT

Alert on safety, not just latency

The scores already exist; nothing watches them. A medical-safety score below threshold, or any call where health data preceded identity confirmation, should page a human the same day. Sampling and a per-call cost budget belong here too, since judges are themselves LLM calls.

THEN

Native answering-machine detection and warm transfer

Voicemail is currently detected by the model calling a tool after it hears a greeting — documented, but slower and less reliable than the detector LiveKit 1.8 now ships. Warm transfer to a human should also carry context, so the care manager does not restart the conversation.

THEN

Treat PHI handling as a first-class feature

Redaction is a flag today and should be the default, with an auditable consent trail per call, retention limits on recordings, and a documented basis for every disclosure. In a real clinical deployment this is what a compliance review asks about first — well before it asks about latency.

SCALE

Concurrency, cost and language

Worker autoscaling with a concurrency cap per number; cost per completed booking as the headline metric rather than cost per call; and Hindi and regional languages, which for an Indian clinic is the difference between a demo and a product.

In short

- **Everything the brief asked for works** and was verified by running it — a real spoken conversation, tool calls, recording attached to the trace, post-call analysis, and two server-side evaluation rules firing on Opik's own infrastructure.
- **The one gap is a telephony account**, not code. Three free tiers were tested and each blocks outbound SIP; the evidence for each is above, and a paid trunk changes one environment variable.
- **The design decisions are defensible** and I am happy to walk through any of them: why the tool record outranks the model, why the conversation is split across traces, why telemetry fails open, and why the Opik module imports nothing from the application it observes.